

Assignment 4

Sai Nikhil Chakravarthi 24cs10018 and nagavamsi 24cs10013

group 28

Section (a): Implementation of 8:1 MUX using 74LS153 and realization of a Boolean function

Aim

1. To implement an 8-to-1 multiplexer using the 74LS153 (dual 4-to-1 multiplexer) IC and minimum additional gates.
2. Using the designed 8-to-1 multiplexer, to implement and verify the 4-variable Boolean function:

$$F(A, B, C, D) = \Sigma(2, 3, 5, 8, 10, 11, 12)$$

Apparatus / Instruments Required

- Digital trainer kit / breadboard with +5V supply
- IC 74LS153 (Dual 4-to-1 Multiplexer)
- NOR gate IC 74LS02(used to implement NOT and OR using only NOR)
- Connecting wires / jumpers
- Switches for inputs and LED for output observation

Theory

Multiplexer

A multiplexer (MUX) is a combinational circuit that selects one of many input lines and connects it to a single output line depending on the select inputs.

An 8:1 MUX has inputs I_0 to I_7 , select lines S_2, S_1, S_0 , and output Y :

$$Y = I_{(S_2S_1S_0)}$$

8:1 MUX Selection Table

The 8:1 multiplexer output is determined by the select lines as shown below.

S_2	S_1	S_0	Output Function
0	0	0	$Y = I_0$
0	0	1	$Y = I_1$
0	1	0	$Y = I_2$
0	1	1	$Y = I_3$
1	0	0	$Y = I_4$
1	0	1	$Y = I_5$
1	1	0	$Y = I_6$
1	1	1	$Y = I_7$

Table 1: Truth table of an 8-to-1 multiplexer

74LS153 IC (Dual 4:1 MUX)

The 74LS153 contains two independent 4-to-1 multiplexers:

- MUX-1: inputs $1C0, 1C1, 1C2, 1C3$ and output $1Y$
- MUX-2: inputs $2C0, 2C1, 2C2, 2C3$ and output $2Y$

Both multiplexers share the same select lines:

$$A \text{ (pin 14),} \quad B \text{ (pin 2)}$$

Each MUX has an **active-low enable** input:

$$\overline{1G} \text{ (pin 1),} \quad \overline{2G} \text{ (pin 15)}$$

Hence, a MUX works normally only when its enable pin is at logic 0. This is an important point: **to enable a block, we must apply 0 to its enable pin.**

Realization of 8:1 MUX using 74LS153

An 8:1 MUX can be constructed using two 4:1 MUX blocks as follows:

- MUX-1 selects among I_0, I_1, I_2, I_3
- MUX-2 selects among I_4, I_5, I_6, I_7

Let the select lines be chosen as:

$$S_0 = C, \quad S_1 = B, \quad S_2 = A$$

The internal select pins of the IC are connected as:

$$\text{pin 14 (A)} \rightarrow C, \quad \text{pin 2 (B)} \rightarrow B$$

The third select line A is used to enable exactly one 4:1 MUX at a time. Since enable pins are active-low, the enable wiring is:

$$\overline{1G} = A, \quad \overline{2G} = \overline{A}$$

Therefore:

- When $A = 0$: $\overline{1G} = 0$ enables MUX-1 and $\overline{2G} = 1$ disables MUX-2.
- When $A = 1$: $\overline{1G} = 1$ disables MUX-1 and $\overline{2G} = 0$ enables MUX-2.

Replacing the 2:1 MUX by OR

In the standard construction, a 2:1 MUX is used to choose between $1Y$ and $2Y$. However, here only one MUX output is active at a time (because of enable logic). Thus, the final output can be obtained using:

$$F = 1Y + 2Y$$

Hence, the 2:1 MUX is replaced by an OR operation.

Using only NOR gates to generate NOT and OR

NOR is a universal gate. Using only NOR gates:

NOT using NOR

$$\overline{X} = X \downarrow X$$

OR using NOR Using DeMorgan's theorem:

$$X + Y = \overline{\overline{X} \downarrow \overline{Y}}$$

First compute:

$$N = X \downarrow Y = \overline{X + Y}$$

Then invert it using NOR:

$$X + Y = N \downarrow N$$

Thus, both NOT and OR can be implemented using only NOR gates.

Implementation of the given function

The function is:

$$F(A, B, C, D) = \Sigma(2, 3, 5, 8, 10, 11, 12)$$

Choose A, B, C as select lines and express F as a function of D . The required 8:1 MUX input assignments are:

$$I_0 = 0, I_1 = 1, I_2 = D, I_3 = 0, I_4 = \overline{D}, I_5 = 1, I_6 = \overline{D}, I_7 = 0$$

Pin Connections (74LS153)

Power Connections

$$V_{CC} \rightarrow \text{pin 16}, \quad GND \rightarrow \text{pin 8}$$

Select Connections

$$\text{pin 14 (A)} \rightarrow C, \quad \text{pin 2 (B)} \rightarrow B$$

Enable Connections (active-low)

$$\text{pin 1 } (\overline{1G}) \rightarrow \overline{A}$$

$$\text{pin 15 } (\overline{2G}) \rightarrow A$$

where:

$$\overline{A} = A \downarrow A$$

Data Inputs

MUX-1 inputs (I_0 to I_3):

$$1C0 \text{ (pin 6)} = I_0 = 0$$

$$1C1 \text{ (pin 5)} = I_1 = 1$$

$$1C2 \text{ (pin 4)} = I_2 = D$$

$$1C3 \text{ (pin 3)} = I_3 = 0$$

MUX-2 inputs (I_4 to I_7):

$$2C0 \text{ (pin 10)} = I_4 = \overline{D}$$

$$2C1 \text{ (pin 11)} = I_5 = 1$$

$$2C2 \text{ (pin 12)} = I_6 = \overline{D}$$

$$2C3 \text{ (pin 13)} = I_7 = 0$$

Output Connection using NOR gates

The two MUX outputs are:

$$1Y \text{ (pin 7)}, \quad 2Y \text{ (pin 9)}$$

Final output:

$$F = 1Y + 2Y$$

Implement OR using NOR:

$$N = 1Y \downarrow 2Y$$

$$F = N \downarrow N$$

Procedure

1. Place the 74LS153 IC on the breadboard/trainer kit.
2. Connect power: pin 16 to +5V and pin 8 to GND.
3. Connect select lines: pin 14 to C and pin 2 to B .
4. Connect enable pins: pin 1 to A .
5. Generate $\overline{A}$ using NOR and connect it to pin 15:

$$\overline{A} = A \downarrow A$$

6. Connect data inputs according to:

$$I_0 = 0, I_1 = 1, I_2 = D, I_3 = 0, I_4 = \overline{D}, I_5 = 1, I_6 = \overline{D}, I_7 = 0$$

7. Generate $\overline{D}$ using NOR:

$$\overline{D} = D \downarrow D$$

and connect it wherever required.

8. Take outputs pin 7 and pin 9 and generate:

$$F = 1Y + 2Y$$

using only NOR gates:

$$N = 1Y \downarrow 2Y, \quad F = N \downarrow N$$

9. Connect F to an LED through a resistor.
10. Vary A, B, C, D using switches and verify output.

Observation Table (Truth Table of F)

The function is $F(A, B, C, D) = \Sigma(2, 3, 5, 8, 10, 11, 12)$. So $F = 1$ for minterms $\{2, 3, 5, 8, 10, 11, 12\}$ and $F = 0$ otherwise.

A	B	C	D	Minterm No.	F
0	0	0	0	0	0
0	0	0	1	1	0
0	0	1	0	2	1
0	0	1	1	3	1
0	1	0	0	4	0
0	1	0	1	5	1
0	1	1	0	6	0
0	1	1	1	7	0
1	0	0	0	8	1
1	0	0	1	9	0
1	0	1	0	10	1
1	0	1	1	11	1
1	1	0	0	12	1
1	1	0	1	13	0
1	1	1	0	14	0
1	1	1	1	15	0

Table 2: Truth table verification for $F(A, B, C, D)$

Result

- The 8-to-1 multiplexer was successfully implemented using the 74LS153 IC and NOR gates.
- The Boolean function

$$F(A, B, C, D) = \Sigma(2, 3, 5, 8, 10, 11, 12)$$

was realized using the MUX method and verified using the truth table.

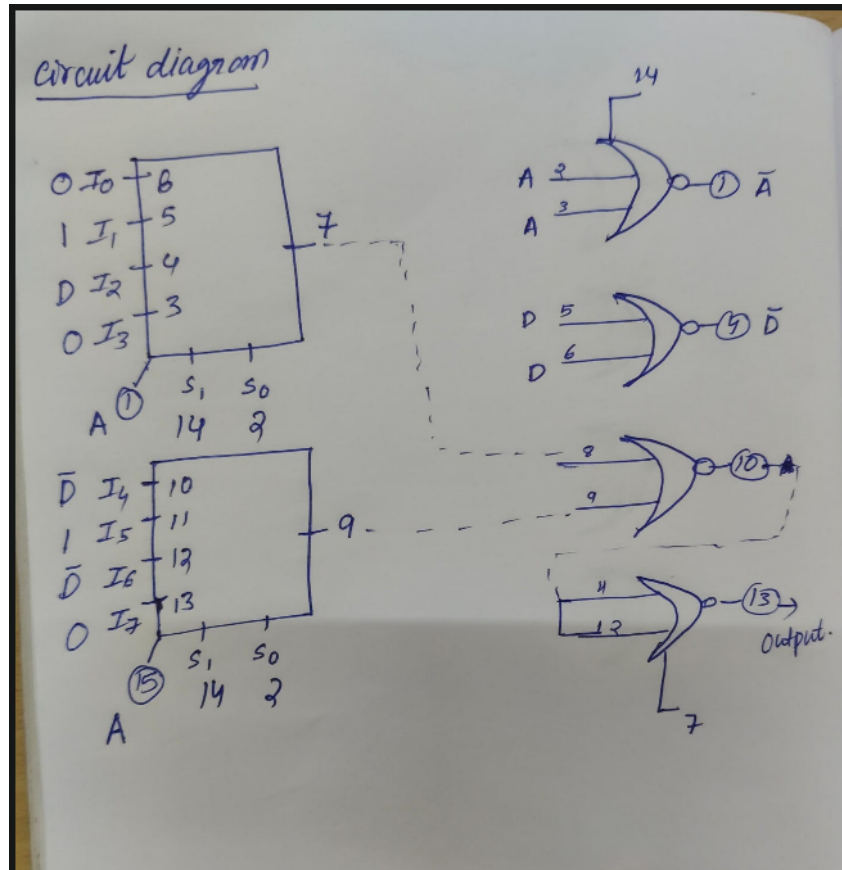


Figure 1: circuit diagram.

Conclusion

The 74LS153 dual 4:1 multiplexer can be used to construct an 8:1 multiplexer by using the enable pins as the third select input. Since the enable pins are active-low, one MUX is enabled at a time. Therefore, the final stage 2:1 MUX can be replaced by an OR operation. Using NOR as a universal gate, both NOT and OR were implemented using only NOR gates, resulting in a minimal hardware realization.

(b) Implementation using 74LS157 (Quad 2-to-1 Multiplexer)

Aim

To design and implement a circuit that selects one out of three 4-bit inputs $A = (A_3, A_2, A_1, A_0)$, $B = (B_3, B_2, B_1, B_0)$, and $C = (C_3, C_2, C_1, C_0)$ based on select lines S_1 and S_2 , and produces the output $F = (F_3, F_2, F_1, F_0)$ according to the given convention.

Apparatus Required

- 74LS157 IC (Quad 2-to-1 Multiplexer)
- Digital trainer kit / breadboard
- +5V DC power supply
- Jumper wires

- Logic switches (for inputs A, B, C, S_1, S_2)
- LEDs with resistors (or logic probe) to observe outputs

Theory

There are three 4-bit inputs:

$$A = (A_3, A_2, A_1, A_0), \quad B = (B_3, B_2, B_1, B_0), \quad C = (C_3, C_2, C_1, C_0)$$

and the output:

$$F = (F_3, F_2, F_1, F_0)$$

The selection convention is:

$$\begin{aligned} S_1 = 0, S_2 = 0 & \quad \because F = A \\ S_1 = 0, S_2 = 1 & \quad \because F = B \\ S_1 = 1, S_2 = X & \quad \because F = C \end{aligned}$$

The IC **74LS157** is a **quad 2-to-1 multiplexer**, meaning it contains four identical 2:1 multiplexers. Each multiplexer selects between two inputs ($I0$ and $I1$) based on a common select line S .

The chip has:

- Four channels: $(1A, 1B) \rightarrow 1Y$, $(2A, 2B) \rightarrow 2Y$, $(3A, 3B) \rightarrow 3Y$, $(4A, 4B) \rightarrow 4Y$
- A common select input S
- An active-low enable input $\overline{G}$ (Enable)

For proper operation:

$$\overline{G} = 0 \quad (\text{enabled})$$

Since the problem requires selecting between **three** 4-bit inputs, we implement it using:

- **Stage 1:** Use one 74LS157 to select between A and B using S_2 .
- **Stage 2:** Use another 74LS157 to select between (output of stage 1) and C using S_1 .

Thus the logic is:

$$F = \begin{cases} A, & S_1 = 0, S_2 = 0 \\ B, & S_1 = 0, S_2 = 1 \\ C, & S_1 = 1, S_2 = X \end{cases}$$

Circuit Design Explanation

Step 1 (MUX-1): Connect A to the “A” inputs of the first 74LS157 and B to the “B” inputs. Use S_2 as the select line. So output of MUX-1 is:

$$M = \begin{cases} A, & S_2 = 0 \\ B, & S_2 = 1 \end{cases}$$

Step 2 (MUX-2): Connect M to the “A” inputs of the second 74LS157 and connect C to the “B” inputs. Use S_1 as the select line. So final output becomes:

$$F = \begin{cases} M, & S_1 = 0 \\ C, & S_1 = 1 \end{cases}$$

Combining both:

$$F = \begin{cases} A, & S_1 = 0, S_2 = 0 \\ B, & S_1 = 0, S_2 = 1 \\ C, & S_1 = 1, S_2 = X \end{cases}$$

Procedure

1. Place two 74LS157 ICs on the breadboard.
2. Connect pin 16 of both ICs to +5V and pin 8 to GND.
3. Connect the enable pin $\overline{G}$ of both ICs to logic 0 (GND) to enable them.
4. **Stage 1:** Connect the 4-bit input A to the A-inputs of IC-1 and B to the B-inputs of IC-1.
5. Connect the select input of IC-1 to S_2 .
6. Take the outputs of IC-1 as an intermediate 4-bit bus $M = (M_3, M_2, M_1, M_0)$.
7. **Stage 2:** Connect M to the A-inputs of IC-2 and connect C to the B-inputs of IC-2.
8. Connect the select input of IC-2 to S_1 .
9. Observe the final outputs F_3, F_2, F_1, F_0 using LEDs/logic probe.
10. Verify outputs for all combinations of S_1, S_2 and different input values.

Truth Table (Selection)

S_1	S_2	Selected Output
0	0	$F = A$
0	1	$F = B$
1	0	$F = C$
1	1	$F = C$

Observation Table

S_1	S_2	Applied Inputs	Observed Output F	Expected Output
0	0	A, B, C	A	A
0	1	A, B, C	B	B
1	0	A, B, C	C	C
1	1	A, B, C	C	C

Observation

When $S_1 = 0$ and $S_2 = 0$, the output F follows input A and changes only when A changes. Changing B or C does not affect F .

When $S_1 = 0$ and $S_2 = 1$, the output F follows input B and changes only when B changes. Changing A or C does not affect F .

When $S_1 = 1$ (irrespective of S_2), the output F follows input C and changes only when C changes. Changing A or B does not affect F .

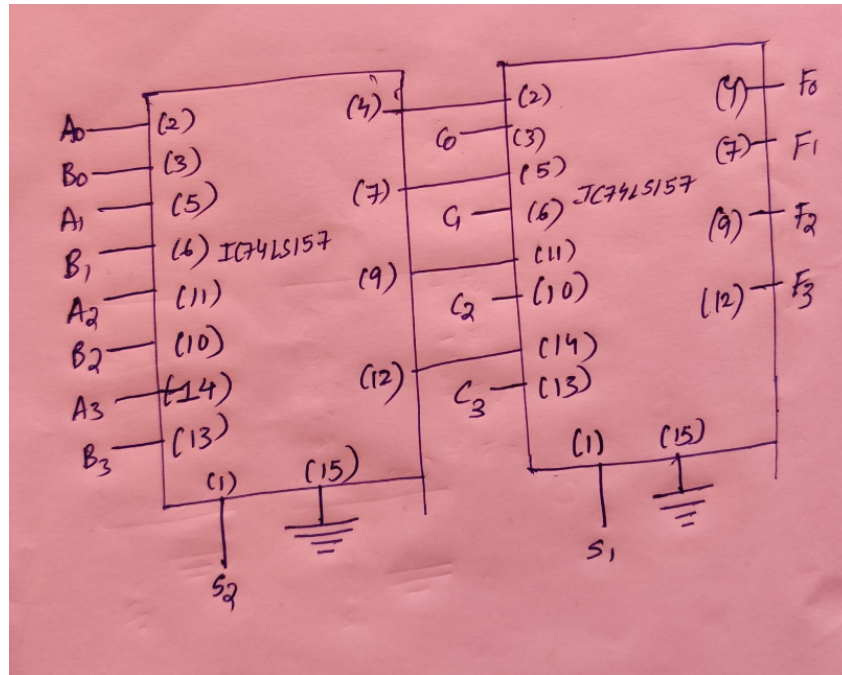


Figure 2: circuit diagram .

Result

The circuit was successfully implemented using two 74LS157 quad 2-to-1 multiplexers. The output F matched the required selection convention:

$$S_1 = 0, S_2 = 0 \Rightarrow F = A, \quad S_1 = 0, S_2 = 1 \Rightarrow F = B, \quad S_1 = 1, S_2 = X \Rightarrow F = C.$$

(c) 4-bit Binary to Hexadecimal 7-Segment Decoder using EPROM 2732

Aim

To design and implement a 4-bit binary to hexadecimal (0 to F) 7-segment decoder using a 4K×8 EPROM IC 2732 by storing the 7-segment codes for the 16 hexadecimal digits in the first 16 memory locations, applying the 4-bit binary input to the four least significant address lines, and verifying the output on a common-anode 7-segment display.

Apparatus Required

- EPROM IC 2732 (4K×8)
- EPROM programmer (to burn data into the EPROM)
- Common-anode 7-segment display
- Resistors (typically 330Ω or 470Ω) for each segment
- Breadboard / Digital trainer kit
- +5V DC power supply
- Jumper wires
- Logic switches for 4-bit input (A_3, A_2, A_1, A_0)

Theory

1. 7-segment display A 7-segment display consists of seven LED segments labelled as:

$$a, b, c, d, e, f, g$$

which can be turned ON in different combinations to display digits and some letters.

In a **common-anode** 7-segment display:

- The anode of all segments is connected to +5V.
- A segment glows when its cathode is driven **LOW (0)**.
- A segment is OFF when its cathode is driven **HIGH (1)**.

Thus, for common-anode displays, the segment control is **active-low**.

2. Why EPROM is used The required circuit is a **4-bit to 7-bit decoder** for hexadecimal digits. Instead of implementing the logic using multiple gates (AND/OR/NOT) or dedicated decoder ICs, an EPROM can be used as a **lookup table (LUT)**.

Advantage of using EPROM:

- **Very simple hardware:** Only the EPROM + resistors + display are needed.
- **Reprogrammable:** If the segment codes change (common-anode/common-cathode, different mapping), we only re-burn the EPROM; no wiring changes are required.

- **Fast and reliable:** Output is directly obtained from stored memory contents.
- **Expandable:** Same EPROM can store patterns for more symbols beyond 0–F.

3. EPROM 2732 working The EPROM 2732 is a **4K×8** memory, meaning:

$$4096 \text{ locations} \times 8 \text{ bits per location}$$

It has:

- 12 address lines: A_0 to A_{11} (select one of 4096 locations)
- 8 output data lines: Q_0 to Q_7
- Enable pin $\overline{E}$ (active-low)
- V_{PP} pin used only during programming

In this experiment:

- Only the first 16 memory locations (0000H to 000FH) are used.
- The 4-bit binary input is applied to A_0 to A_3 .
- All higher address bits (A_4 to A_{11}) are tied to logic 0.

So the EPROM acts as:

$$(A_3A_2A_1A_0) \rightarrow (Q_6Q_5Q_4Q_3Q_2Q_1Q_0)$$

where outputs represent segment control bits.

EPROM Program Data (Hex File)

The 7-segment codes for hexadecimal digits 0 to F were stored in EPROM locations 0000H to 000FH. The hex bytes programmed into the EPROM are:

Address (Hex)	Input ($A_3A_2A_1A_0$)	Data Byte (Hex)
0000	0000	01
0001	0001	4F
0002	0010	12
0003	0011	06
0004	0100	4C
0005	0101	24
0006	0110	20
0007	0111	0F
0008	1000	00
0009	1001	04
000A	1010	08
000B	1011	60
000C	1100	31
000D	1101	42
000E	1110	30
000F	1111	38

Note: These bytes correspond to the segment ON/OFF pattern for a **common-anode** 7-segment display.

Pin Connections of EPROM 2732

Power and control pins

- Pin 24 (V_{CC}) $\rightarrow$ +5V
- Pin 12 (V_{SS}) $\rightarrow$ GND
- Pin 18 ($\overline{E}$) $\rightarrow$ GND (Enable always active)
- Pin 20 ($V_{PP} / \overline{G}$) $\rightarrow$ +5V during normal read operation

Address lines

- Pin 8 (A_0) $\rightarrow$ Input bit A_0 (LSB)
- Pin 7 (A_1) $\rightarrow$ Input bit A_1
- Pin 6 (A_2) $\rightarrow$ Input bit A_2
- Pin 5 (A_3) $\rightarrow$ Input bit A_3 (MSB)

All higher-order address lines are grounded:

- A_4 (Pin 4), A_5 (Pin 3), A_6 (Pin 2), A_7 (Pin 1), A_8 (Pin 23), A_9 (Pin 22), A_{10} (Pin 19), A_{11} (Pin 21) $\rightarrow$ GND

Data output lines to 7-segment EPROM output pins:

$$Q_0, Q_1, Q_2, Q_3, Q_4, Q_5, Q_6, Q_7$$

are available on pins:

$$9, 10, 11, 13, 14, 15, 16, 17$$

Connect each EPROM output through a current-limiting resistor to the corresponding 7-segment pin.

Example mapping (one possible standard mapping):

EPROM Output	EPROM Pin	7-segment Segment
Q_0	9	a
Q_1	10	b
Q_2	11	c
Q_3	13	d
Q_4	14	e
Q_5	15	f
Q_6	16	g
Q_7	17	(DP not used / optional)

The common anode pin of the display is connected to +5V.

Procedure

1. Program the EPROM 2732 using the EPROM programmer with the given hex data bytes at addresses 0000H to 000FH.
2. Place the EPROM 2732 on the breadboard/trainer kit.
3. Connect V_{CC} (Pin 24) to +5V and V_{SS} (Pin 12) to GND.
4. Connect $\overline{E}$ (Pin 18) to GND to enable the EPROM output.
5. Connect V_{PP} (Pin 20) to +5V for normal read mode.
6. Connect the input switches to address lines A_0 – A_3 .
7. Tie all higher address lines A_4 – A_{11} to GND.
8. Connect EPROM outputs Q_0 – Q_6 to the 7-segment display segments a – g through current-limiting resistors.
9. Connect the common-anode terminal of the 7-segment display to +5V.
10. Apply all 4-bit input combinations from 0000 to 1111 and verify that the display shows hexadecimal digits 0 to F correctly.

Observation Table

Input (Hex)	Input ($A_3A_2A_1A_0$)	EPROM Data (Hex)	Displayed Digit
0	0000	01	0
1	0001	4F	1
2	0010	12	2
3	0011	06	3
4	0100	4C	4
5	0101	24	5
6	0110	20	6
7	0111	0F	7
8	1000	00	8
9	1001	04	9
A	1010	08	A
B	1011	60	b
C	1100	31	C
D	1101	42	d
E	1110	30	E
F	1111	38	F

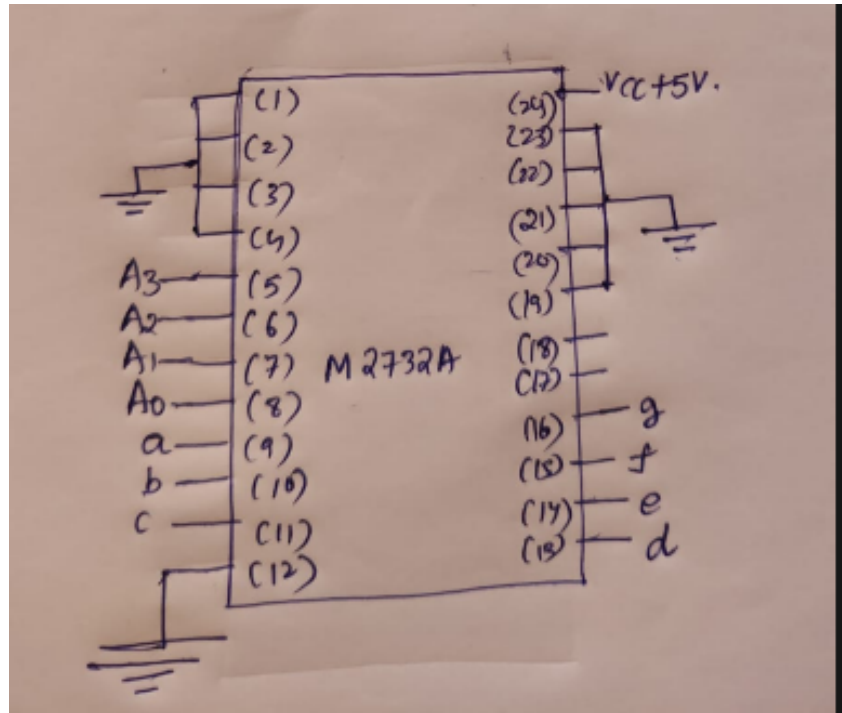


Figure 3: circuit diagram .

Result

A 4-bit binary to hexadecimal 7-segment decoder was successfully designed and implemented using EPROM 2732. The 7-segment codes for hexadecimal digits 0 to F were stored in the first 16 EPROM locations. By applying the 4-bit input to address lines A₀–A₃ and grounding higher address lines, the EPROM acted as a lookup table and correctly displayed digits from 0 to F on the common-anode 7-segment display.